

Distributed Systems - 24_11

Software Replication in Synchronous Systems

We'll see how to replicate elements, including an entire system. In order to replicate elements, we need to define a criteria of consistency, and we will use linearizability.

Let's see an example: We have a list, and clients executing operations on that list, like adding an element to it, or getting an element from the list. We built a distributed object, with two clients interacting with the same list. What is the drawback with this solution? If the server with the list crashes, the object is lost forever. Let's say we have p probability of failure, how can we increase the reliability of the object. The idea is to use replication. By replicating the object on several servers, I decrease the probability of the object failing (which means that all servers would go down). We'll see how we can replicate our object on k servers, tolerating $f = k - 1$ failures.

Let's say now that the probability of failure for each server is the same p , so what is the probability of the object failing? If we have $p = \frac{1}{2}$ and three servers, then the probability of the object failing would be $\frac{1}{2^3} = \frac{1}{8}$. This shows that, even if we have something that is unreliable, by replicating the object we can achieve something that works.

The motivation behind replication is to have high reliable objects. The probability of the object working will be given by $1 - p^N$. We are simplifying our computation by assuming that the probability of a server failing is independent. In real life, how can we ensure independence? With diversity: we have to make each replica different from the other. If we use the same operating systems for the replica, and there is a vulnerability for one, then it's spread to all else. To fight this problem, we have to use different operating systems, softwares, and even location.

Let's look now at the system model. We'll have a set of clients, and a set of other processes which are the replicas. A replica is simply a copy of our object. C is the number of clients, N the number of replicas.

A client will interact with our object by a set of operations. These operations will either read the state of the object, or modify it.

For example, given the object as a list, we could have

object.operation(parameters) client_num. The operation will be executed, and will return a reply to the client, like *L.OK() C₁*. Our assumption is that the same client does not invoke a new operation on the same object until the

operation he has already invoked completes.

These objects are not implemented by a single server, so it will be implemented behind N replicas. We have to build a protocol in such a way that operations are transparent to the client, who does not need to know how many replicas there are, and so on.

When a client issues an operation, we will have the replicas implement the operation for the client. The problem occurs when there are several clients operating with the object at the same time. Thus, we need to define a consistency semantic for our object, which specifies what will be the value of the operation in a system where we have concurrency. It will define what are the acceptable runs. There are three main criteria:

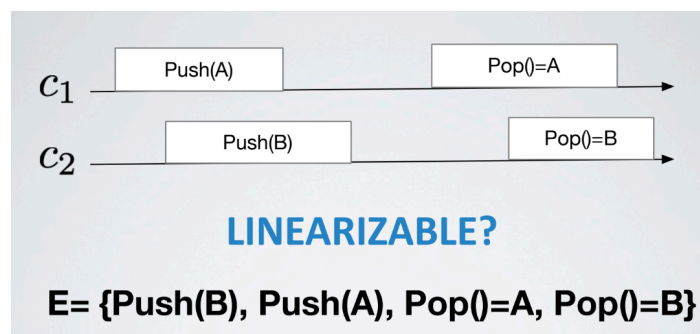
- linearizability: same definition of the registers
- sequential consistency
- causal consistency: operations respect the causal order.

We will only take a look at linearizability.

An execution on the object o is linearizable if for any operation Op on o we can find a fictional point between the start and the end of Op in which the operation runs instantaneously and is visible to our entire system.

It's a similar definition to registers.

Let's see an example with a stack:



How do we implement a generic linearizable replicated object?

There are some sufficient conditions for linearizability, we can implement it if we are able to make the system of replica behave in a certain way. We need:

- atomicity of operations: all replicas have to handle the invocation, if one replica handles it.
- ordering: if replica 1 executes op_1 and then op_2 , then a second replica also has to execute op_1 before op_2 .

- blocking operations: the client does not see the result of an operation until at least one (correct) replica handled it.

These conditions are sufficient but not necessary.

Notice that ordering and atomicity reminds us of total order. So, to implement this consensus is necessary.

There are two main ways of implementing this scheme.

The first technique is known as active replication, by which the client and replica run more or less the same software.

We'll use a UTOB running on the client and each of the replicas. When we are the client, we send the request via UTOB, and if you are the replica...

The TOB ensures that everyone sees the same order of operations.

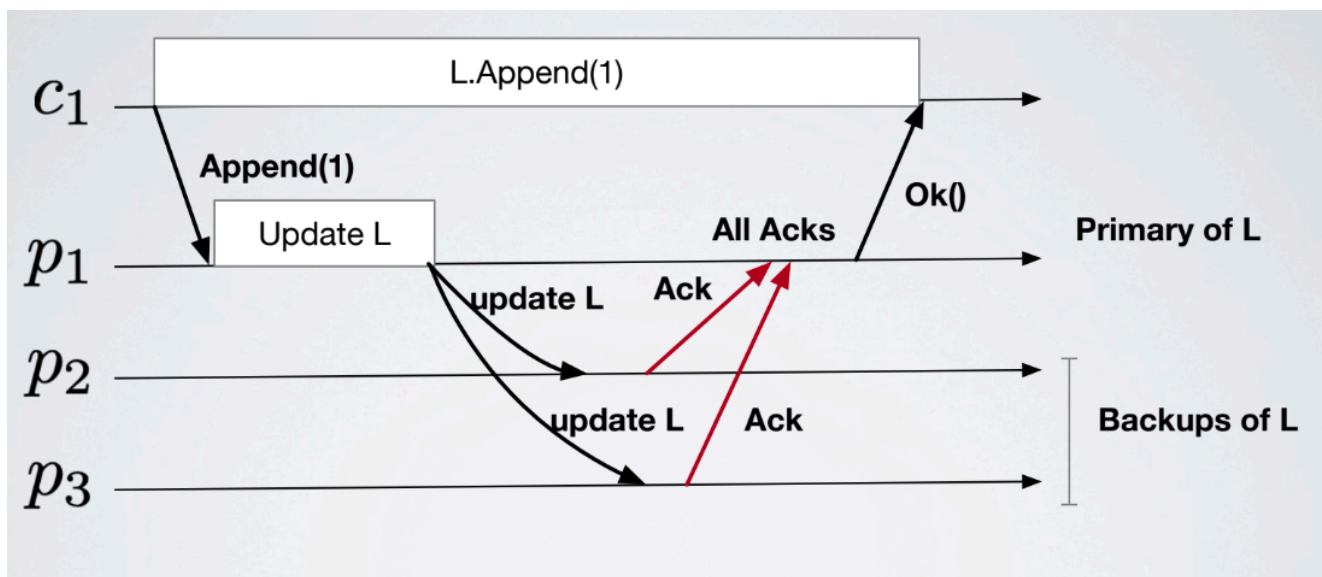
For example we could have one client and three replicas. Then each replica will have a set of ordered operations, and each replica will have the same order, this because of TOB. There is a problem with this approach, given by non-deterministic operations, and since we're running the TOB both on clients and replicas, this approach does not scale. So, usually we can use this approach only with small systems.

So, the standard is given by the passive replication approach, also known as Primary Backup. Most of the replicas will be passive, following a special replica elected as the leader. The client will just interact with the primary, and the primary has the job to make all the other replicas consistent, and once this happened he will reply to the client. This is scalable as the increased sending of messages only relates to the replicas.

In the primary backup approach, the replicas are partitioned into two set: primary (a singleton) and the backups. The primary is unique, and has the job to receive the invocation from the client, and send back the answer. The backups only interacts with the primary, and used only when there is a failure.

We'll use P to implement this.

Let's say first we don't have failures, starting with an example:

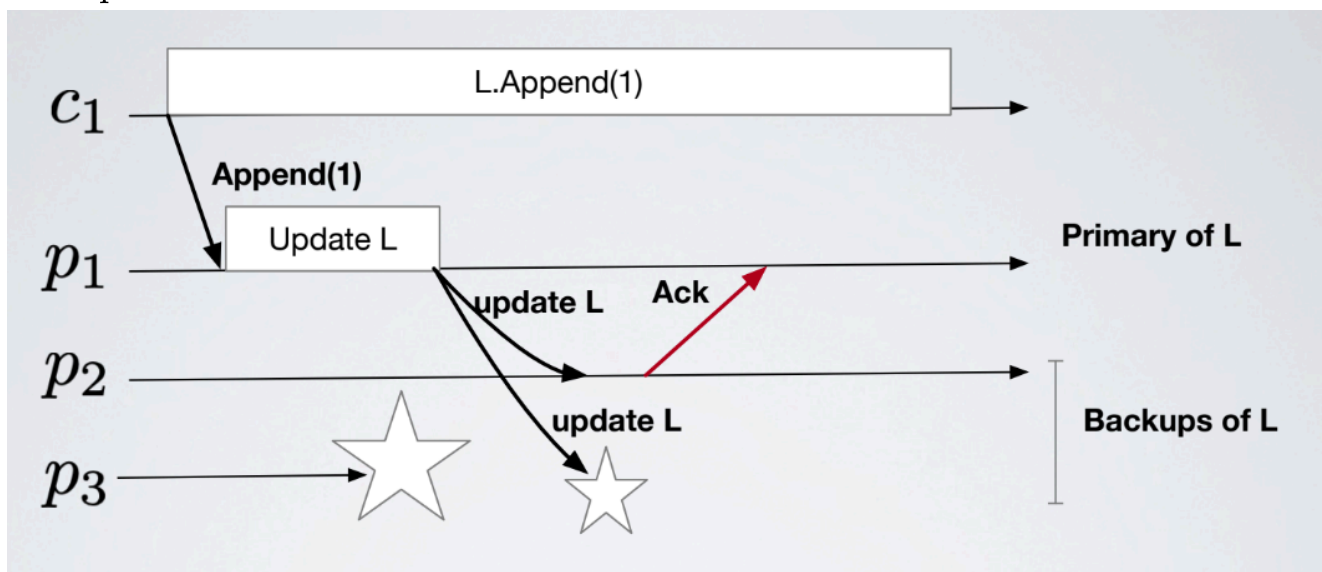


Each client knows the identity of the primary. The client will send a request to the primary, who will not send the answer to the client immediately after finishing the execution, rather it will communicate with the other replicas, and wait for ACKs coming from them. Only then it will answer the client.

Is there atomicity in this pattern? If there are no failures, then we have all the backups execute an operation. Is there ordering? Yes, as the primary serializes the operations.

In the case of no crashes, the algorithm is trivial.

What happens if there is a crash? Let's start by taking a look at the case where a backup crashes.



In this case, the primary has to wait until he receives the acks from the corrects.

What happens if the primary crashes? We can have three scenarios:

1. The primary fails after completing operations, and there is no pending operations, with the client having already received the answer.

2. The primary fails after receiving all the ACKs, but before sending the response to the client.
3. The primary dies having only some of the replicas updated, thus leaving the system in an inconsistent state.

In all three cases, we have to elect a new primary.

Let's take a look at all scenarios.

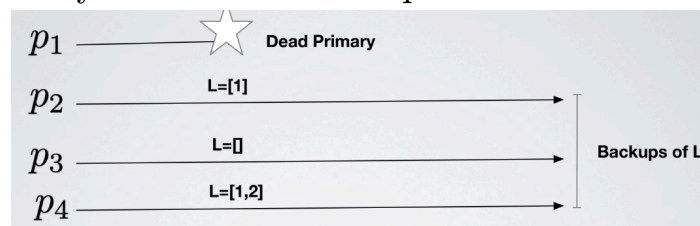
In the first scenario, we have to run a leader election algorithm between the replicas, electing one of them, and when the client wants to perform a new operation, he will communicate with the new leader.

In the second scenario, we run a leader election algorithm, electing a new primary. The client, after not receiving anything for a certain time, will send the request again. However this could be problematic, as the request will take place twice (which we don't want that). How do we fix this? We use a timestamp, with (c_n, r_n) , where c_n is the client number, and r_n is the request number, followed by the executed operation.

We are making requests unique, but in order to make this strategy work, we have to keep track also of a list of commands executed on the object.

In the third scenario, we have the primary failing in the middle of updating the backups, so some backups will be updated, others not. In this case, we need to have replicas synchronize. To do so, we can have the newly elected primary asking to every other process what they have, and only then he starts answering to clients. Now, when we run the leader election, it's not enough that we take the maximum ID, it's better that if I always elect the backup that is in the newest state.

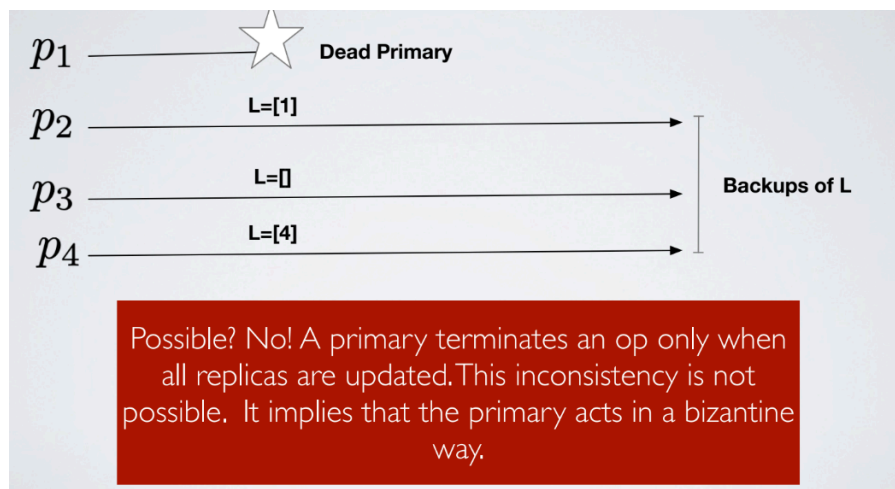
Now, is it possible that, given a list in which we only have append and read operation, I freeze the system and see the replicas are in the following state:



No, as by having 2 in a list, we have to have that every list has 1 inside. This cannot happen because the leader cannot finish appending 1, before appending 1 has finished.

This implies that the primary executes $\text{append}(2)$ when $\text{append}(1)$ was not completed yet.

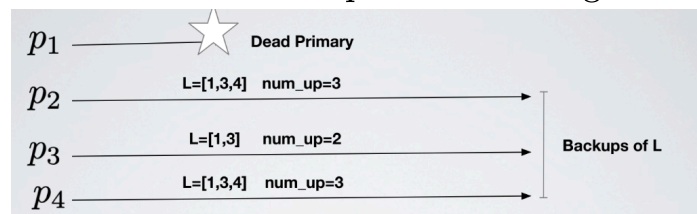
Also this execution is not possible:



The only possibility when a primary dies is having a partition of them in two sets. We'll have the updated set, having the latest operation executed, or the non-updated set.

In every replica we'll keep a counter of operations, such that if the primary dies the maximum difference between counters will be 1.

For this example, we'll have to elect the process with highest number:



Here is the code:

```

upon event INIT
  primary =  $\perp$ 
  deadprimary =  $\perp$ 
  Crashed =  $\emptyset$ 
  update_counter = 0
  ACKS = [ $\emptyset, \emptyset, \dots, \emptyset$ ] ▷ size equal to  $|\Pi|$ 

upon event CRASH(p)
  Crashed = Crashed  $\cup$  {p}

upon event primary  $\in$  Crashed
  BebCast(< ELECTION, primary, myID, update_counter >)
  primary =  $\perp$ 

upon event BEB DELIVER(< ELECTION, IDDEADPRIMARY, ID, CNT > from p)
  if primary == IDDEADPRIMARY then
    Crashed = Crashed  $\cup$  {primary}
    BebCast(< ELECTION, primary, myID, update_counter >)
    primary =  $\perp$ 
    ACKS[IDDEADPRIMARY] = ACKS  $\cup$  {< ELECTION, IDDEADPRIMARY, ID, CNT>}

upon event EXISTS j SUCH THAT ACKS[j]  $\cup$  Crashed =  $\Pi$ 
  primary = take the lowest ID with the highest counter in ACKS[j]
  ACKS[j] =  $\emptyset$ 

```

When we detect the primary as dead, we send a ELECTION message proposing ourselves, sending the id of the primary we want to replace, our ID and counter.

Once another process receives this message, we collect in the ACKs set all the messages for the leader election to replace a certain process. I terminate when I received an ACK from every correct process.

Once the new primary is elected, he will send to everyone in the system the state of the object, so that they are updated. With this we guarantee the atomicity. Once the new primary receives ACKs from everyone, he will send the confirmation of operation to the original client, and start serving new requests.

If we, as a primary, fail before having any other replica receive the new value, are we guaranteed that the algorithm handles this case? Yes.

Let's now take a look at a question.

▪Let us say that I give you the task of implementing a generic distributed object O , in an asynchronous systems with $n=10$ processes and $f=1$ failure. If you succeed in the task I will give you 100 dollars. Will we gain the money? No, as if we managed to use this object we would be able to solve consensus in asynchronous system.

In fact, when a value is proposed we could append the value to the object. Once the value in the list is not null anymore, we'll decide the first value.

...

What are the positive points of primary backups?

- It's easy to understand
- It works even if executions are not deterministic, as we have a single primary executing this kind of operations (like getting a random value, which will be disseminated later to every backup).
- It's easy to add backups quite easily

We also have negative points:

- We have a single point in the system, so if a primary has thousands of requests, the primary has to be fast enough to satisfy them.
- Crash in the middle of updates it's fairly hard to solve in eventually synchronous systems.

In the active replication, in the other hand, there is no primary, but there are problems like non-scalability of the system, and non deterministic executions are not handled.

It will be seen better on Wednesday

Let's now see an exercise:

Show that any implementation of UTO is sufficient to create a (non-fault tolerant) mutual exclusion algorithm. How would you modify your algorithm to have a fault-tolerant mutual exclusion algorithm given the P failure detector.

For the first part, we know that we already have TOB, and by using Broadcast and Deliver event, we have to solve mutual exclusion.

We need to determine the handlers that we need.

Let's say that I have to make a request to the critical section, telling others that we want to get in. If it's not free, processes will have to remember that I want to get in.

- Request()
 - TOB(<REQUEST, id>)

Then, we'll have something else in our algorithm:

We'll have a list, cslist, containing the requests for the Critical Section.

- CSLIST=[]
- TOBDeliver(<REQUEST, id>)
 - CSLIST.append(id)
- upon event CSLIST[0] = *self*
 - CS=True
- Release() if(c==True) #if we are in the critical section
 - TOB(<RELEASE, id>)
- TOBDeliver(<RELEASE, id>)
 - Release = Release $\cup$ {*id*} #we have a release set containing the released set
- upon $x = L[0] \in$ Release
 - $L = L[1 :]$ #remove head of the list
 - Release = Release / {*x*}

This algorithm is not fault tolerant. If there is a failure, it's possible that a process enters the CS, doesn't send the release, and locks everyone else. With P we check if $L[0]$

is in the released set or is dead, if it's dead we can't do much.